

제24회 임베디드SW경진대회 개발계획서

[자동차/모빌리티]

□ 개발 개요

○ 요약 설명

선택 주제	차량 내부 및 외부 정보를 활용한 사용자 체감 솔루션 개발
팀 명	쿠글
작품명	능동형 스마트 글라스 모빌리티
작품설명 (3줄 요약)	- 전/후방 카메라·내부온도·외부 날씨·외부 조도 정보를 활용, 자체 설계 알고리즘을 통해 상황을 능동 판단하여 1:10 아이오닉5 차량 모형의 유리 투명도를 실시간 독립제어한다. - 자체 설계 전력 구동부와 SPWM생성, MI 제어 방식을 도입하여 PDLC의 투명도를 정밀 독립 제어한다. - 열부하 경감, 차박 프라이버시 보호, 카메라 과포화 완화라는 사용자 체감 가치를 제공한다.

○ 개발 배경 및 동기

- 모든 차량에는 유리가 사용된다. 차량 유리는 단순한 유리가 아니다. 차량의 내부와 외부를 분리함과 동시에 연결하는 인터페이스이다. 전기차·자율주행 시대의 차량 유리는 단순 바람 가림막이 아닌 능동 광학 인터페이스로 진화해야 한다.
- 자율주행, 주행 보조 기능 도입으로 인해 카메라·센서 의존도가 높아지는 모빌리티 환경에서 강한 역광·직사광은 운전자의 시야 위협뿐만 아니라 카메라 입력 과포화 문제도 일으킨다.
- 하절기 일사에 의한 차량 내부 열부하는 승차 시 불쾌감을 키우고, 전기차 냉방 전력 소모를 증가시켜 주행거리에 영향을 미친다.
- 차박 문화와 도심 주차 환경에서 차량내 프라이버시 확보에 대한 요구가 있다.

○ 개발 목적 및 목표

- MI 제어 기반 능동형 스마트 글라스 플랫폼으로 위 문제들을 단일 시스템에서 해결한다.
- 자체 제어 알고리즘 개발, H-Bridge + LC Filter + SPWM 구조의 자체 설계 전력 구동부로 다채널 독립·연속 투명도(MI) 제어를 구현한다.
- 열부하 경감, 차박 프라이버시 보호, 카메라 과포화 완화라는 세가지 사용자 체감 가치를 태블릿 UI 기반 시연 및 시연 영상을 통해 입증한다.

○ 작품 상세 설명

- 물리 시연 작품은 1:10 아이오닉5 도면 기반 3D 프린팅 조립 모형 1대로, 전면 2채널·도어 4채널·후면 1채널·선루프 1채널의 8채널 PDLC로 구성된다.
- Raspberry Pi가 상황 인지와 목표 MI값 산출을 담당하고, ESP32 2개가 실시간 SPWM 전력 제어를 담당한다.
- 어느 위치의 유리를 어느 정도 산란시킬지 상황에 따라 능동 판단한다.

□ 개발 방향 및 전략

○ 개발 방향

- MVP 확장 방식: 작품의 완성 목표는 PDLC 8 채널 독립 제어 시스템이다. 이미 직접 설계, 테스트 된 1 채널 전력 회로를 바탕으로 단계적 구현을 진행한다. 신규 회로 위험을 통제하기 위해 검증된 단일 채널 전력 보드(Power Stage PCB)를 2 채널 MVP 에서 먼저 검증한 뒤, 동일 설계를 8 장으로 복제해 8 채널로 확장한다.
- 모듈화 설계: Carrier PCB 와 Power Stage PCB 를 분리, 모듈화를 통해 개발 risk 를 분산한다. 하나의 부품이 고장나더라도 교체가 용이하게 한다. 고전압부와 저전압부를 분리하여 안전 위험을 관리한다.
- 역할 분리 구조 : Raspberry Pi 5 가 영상처리·센서융합·외부정보·정책 판단·UI·로그를 담당하고, ESP32-S3 2 개가 16kHz SPWM 실시간 전력 제어와 Fault 로컬 차단을 담당한다. Raspberry Pi 에서 control 서비스와 UI 서비스를 프로세스 수준으로 분리해 UI 장애가 안전 제어에 영향을 주지 않는 구조를 채택한다.
- 3 계층 능동 보정 제어 : 영상 페루프 기반 L1 제어, 방향성 조도·환경 정보 기반 L2 융합, 실험 로그 기반 L3 보정을 결합하여 PDLC 의 목표 MI 를 연속 산출한다. 축적된 영상·조도·온도·MI 로그는 오프라인 분석을 통해 LUT 와 제어 파라미터 보정에 활용한다.
- 센서 전단 광학 보완: 카메라 노출은 고정하지 않는다. 카메라 자동노출(AE)을 대체하지 않고, AE 가 감당하기 어려운 순간적인 아주 강한 역광 구간에서만 센서 전단 유리부에서 입력광 peak 를 물리적으로 완화하는 보완 계층으로 동작한다.
- 안전 설계 내재화: ESP32 로컬 Fault 즉시 차단, E-Stop NC 회로, 고전압부/저전압부 PCB 분
리, 72V DC Link + 퓨즈·방전저항·HV 표시 LED 다중 보호를 통해 안전을 확보한다.
- 시연 신뢰성 우선 : 태블릿 단일 화면 중심으로 핵심 시연을 보여주며, control/UI 서비스 분리·mock/replay 실출력 차단·LIVE 배지로 시연 정직성과 안정성을 확보한다.

○ 개발 방법 및 활용 기술

- SW분야

- 영상처리·카메라 입력

- 사용 기술: Python, OpenCV, libcamera/picamera2
- 공개 SW: OpenCV(Apache-2.0), libcamera(LGPL 계열), picamera2(Raspberry Pi 카메라용 libcamera 기반 Python API), NumPy.
- 기존 구현 검토: Raspberry Pi camera examples, OpenCV threshold/HSV/edge detection 예제, 자동차 카메라 AE/HDR 보정 사례.
- 자체 구현: camera_service.py, roi_analyzer.py, front_glare_estimator.py, camera_evidence_logger.py.
- 사전 준비: 카메라 AE를 켜 상태에서 강광 baseline 확보, PDLC 산란 미개입/개입 비교 영상 촬영, ROI별 포화 면적 감소와 Edge 보존 여부 검증.

- 방향성 조도·센서융합

- 사용 기술: VEML7700 × 4(전·후·좌·우), TCA9548A I2C Mux, I2C scan
- 공개 SW/기존 구현: VEML7700 Python/Arduino 예제, TCA9548A I2C multiplexer 예제, I2C scanner, Adafruit/SparkFun 계열 센서 드라이버 참고.
- 자체 구현: directional_lux_service.py, lux_calibrator.py, lux_vector_fusion.py.
- 사전 준비: 센서 4개 동일 주소 충돌 해결, 손전등 360° 회전 시 θ_{lux} 추정 정확도 확인, 전/후방 카메라 highlight cue와 조도 벡터 융합 검증.

- 외부정보 수집

- 사용 기술: Open-Meteo API 또는 공공데이터포털 기상 API, REST client, 저주기

cache.

- 수집 항목: 기온, 구름량, 강수, UV, weather stale flag.
 - 공개 SW: requests/httpx, Open-Meteo API, JSON parser.
 - 자체 구현: weather_cache.py, external_context.py.
 - 사전 준비: API 호출 주기 제한, 네트워크 실패 시 마지막 cache 사용
- 정책·제어 알고리즘

- 사용 기술: rule-based ChannelScore, MI-Vrms-투과도 LUT inverse, MI Servo.
- 자체 구현: policy_engine.py, channel_score.py, scenario_manager.py, lut_mapper.py, mi_servo.py.
- 사전 준비: MI sweep 실험, MI-Vrms-투과도/영상지표 LUT 작성, hysteresis·rate limit·low-pass·minimum hold time 튜닝.
- 시스템·UI·로그
 - 사용 기술: systemd, Flask, Flask-SocketIO, pyserial, MJPEG, React, TypeScript, Vite, React Three Fiber, three.js.
 - 공개 SW: Flask(BSD 계열), Flask-SocketIO(MIT), pyserial(BSD), React/Vite/R3F/three.js(MIT).
 - 자체 구현: StateStore, CommandQueue, command_validator.py, mjpeg_proxy.py, /demo 3D HMI, LIVE/REPLAY/MOCK 배치.
 - 로그: CSV + run_meta로 timestamp, scenario, ROI 지표, AE metadata, lux vector, temperature, weather, target_mi, telemetry, fault 기록.
- ESP32 실시간 펌웨어·통신
 - 사용 기술: ESP-IDF, FreeRTOS, MCPWM, USB Serial 20Hz JSON Lines.
 - 구동 방식: 16kHz carrier 기반 Simplified Unipolar SPWM, 60Hz sine-wave equivalent 출력, MI 기반 Vrms 제어.
 - 공개 SW/기존 구현: ESP-IDF MCPWM examples, FreeRTOS task 구조, JSON Lines parser 예제, SPWM/H-Bridge inverter 예제, open inverter/micro-inverter 구동 개념.
 - 자체 구현: serial_parser, channel_manager, spwm_generator, fault_manager, telemetry_reporter.
- 테스트·검증 자동화
 - 사용 기술: pytest, mock serial, ESP-IDF Unity 후보, HIL checklist, fault injection.
 - 공개 SW: pytest(MIT), GitHub Actions 또는 로컬 CI, ESP-IDF test framework.
- HW분야
 - 메인·실시간 제어부
 - Raspberry Pi 5: 영상처리, 센서융합, 날씨 API, 정책 판단, UI 백엔드, CSV 로그 담당.
 - 사전 준비: Raspberry Pi OS 64-bit Lite 설치, I2C/Camera/Serial 활성화, ESP32 #A/#B USB 포트 고정, ESP-IDF 빌드·플래시 환경 구축.
 - PDLC 전력 구동부
 - 사전 준비: PDLC 면적·정전용량·소비전류·응답시간·투과도 실측, MI-Vrms-투과도 LUT 작성, 저전압 터미 부하 검증 후 PDLC 연결.
 - PCB·배선 구조
 - 개발 전략: CH0/CH1용 Power Stage PCB 2장으로 MVP 검증 후 동일 보드를 최종 8장 복제.
 - 사전 준비: 핀맵 확정, 커넥터 극성·실크 라벨·test point 정의, 고전압/저전압 영역 분리, 절연거리·퓨즈·방전저항 검토.
 - 센서·시연 장치
 - 센서: 전방 카메라, 후방 카메라, VEML7700 × 4, TCA9548A, DS18B20 내부온도, 전압/전류 센서, MOSFET/인덕터 온도센서.
 - 시연 장치: 고휘도 LED/손전등, 뜨거운 여름 시나리오용 광원·열원, 태블릿 UI, 로컬 Wi-Fi 또는 travel router.
 - 사전 준비: PDLC 차단·전극 처리, 센서 위치 보정, 차광 후드 제작, 고전압부 제어박스 격리, PDLC 출력선과 센서선 분리 배선.
 - 안전 설계
 - 하드웨어 차단: NC E-Stop, EN_GLOBAL, 채널별 EN_LOCAL, 메인 퓨즈, 채널별 퓨즈, 방전저항, HV LED.
 - 펌웨어 차단: TTL timeout, fault latch, 과전류/과전압/과열 local shutdown.
 - 운영 안전: 고전압부 닫힌 박스 배치, 커넥터 라벨링, 계측 전 방전 확인, 시연 중 fault 발생 시 출력 차단 우선.
 - 사전 준비: E-Stop 동작 시험, fault clear 절차, 통신 단절 시험, 과열 모사 시험, 안전

체크리스트 작성.

- 공개 SW·기존 구현 조사 목록

- 영상/카메라: OpenCV, libcamera, picamera2, Raspberry Pi camera examples, OpenCV ROI/threshold/edge detection 예제.
- 센서: VEML7700 예제 코드, TCA9548A I2C multiplexer 예제, I2C scanner, DS18B20 OneWire 예제.
- 외부정보: Open-Meteo API, 공공데이터포털 기상 API, requests/httpx 기반 REST 예제.
- UI/서버: Flask, Flask-SocketIO, pyserial, MJPEG streaming 예제, React, TypeScript, Vite, React Three Fiber, three.js, drei.
- 펌웨어: ESP-IDF, FreeRTOS, MCPWM examples, ESP-IDF Unity, ESP serial monitor.
- 전력제어: SPWM inverter 예제, H-Bridge gate driver application note, LC filter 설계 예제, open inverter/micro-inverter 구현 사례.
- 선행 기술: 자동 감광 미리, 고정형 선티 필름, 나노 쿨링 필름, PDLC 전압가변 투과도 연구, 차량용 스마트 윈도우 PWM 제어 회로 연구.
- 테스트/배포: pytest, mock serial, GitHub Actions, systemd service, CSV logging/replay 구조.
- 라이선스 관리: 모든 공개 SW의 버전·라이선스·원문 링크·수정 여부를 THIRD_PARTY_NOTICES.md에 정리하고, 제출용 GitHub 공개에 대비

- 개발 환경

- 공통 개발

- macOS, VS Code, GitHub, GitHub Issues/Projects, README, docs, calibration, logs 디렉터리 관리.
- 저장소 구조: raspberrypi/, firmware/, ui/, pcb/, docs/, calibration/, logs/, scripts/.
- 설정 파일: requirements.txt, package.json, sdkconfig.defaults, config.yaml, carrier_pinmap.yaml, .env.example.

- Raspberry Pi 환경

- Raspberry Pi 5, Raspberry Pi OS 64-bit Lite, Python venv, OpenCV, picamera2/libcamera, Flask, Flask-SocketIO, pyserial, pytest.
- I2C 활성화, Camera 활성화, udev alias로 ESP32 #A/#B serial port 고정.
- systemd 배포: kuglass-control.service, kuglass-ui.service, 자동 재시작, 로그 확인.

- ESP32 환경

- VS Code + ESP-IDF extension, ESP-IDF version 고정, ESP32-S3-DevKitC-1 × 2.
- MCPWM 16kHz carrier, 60Hz sine table, MI update, serial_parser, fault_manager 구현.
- flash/monitor script, pinmap header, telemetry viewer 준비.

- UI 환경

- Node.js, pnpm 또는 npm, TypeScript, Vite, React, React Three Fiber, three.js.
- 태블릿 가로 화면 기준 /demo 화면, 로컬 Wi-Fi 접속, offline mode 준비.

- 회로·기구 환경

- KiCad 또는 EasyEDA, 회로도/PCB/Gerber/BOM 관리. 3D 모델링/프린팅 도구, PDLC 재단 도구, 센서 후드 출력.
- 오실로스코프, 멀티미터, 온도계/열화상 카메라, 더미 부하, 퓨즈, 방전저항 준비

- 검증 환경

- pytest + mock serial 자동 테스트.
- HIL: Raspberry Pi - ESP32 - Carrier PCB - Power Stage PCB - PDLC 실제 연결 시험.

○ 유사 작품(제품)과의 비교 분석 및 차별성

구분	기존 방식의 한계	본 작품의 차별성
기존 스마트 글라스	ON/OFF 일괄 수동제어	연속 제어 + 위치별 독립·능동 제어
상용 PDLC 드라	외부 전원 모듈 의존, 수동 버튼	자체 H-Bridge + LC SPWM 전력부, 연

구분	기존 방식의 한계	본 작품의 차별성
이버	으로 ON/OFF만 가능	속 제어 가능, 능동 제어
카메라 후처리 (HDR/AE)	센서 내부 처리, 초강광 영역 잔존	센서 전단 광학 계층으로 AE 보완
수동 가림막	차박시 수동 조작	상황에 따라 전 채널 자동 차광
기존 선텁	상황별 능동 조절 불가	임베디드 알고리즘을 통해 능동 제어

○ 예상되는 장애요인 및 해결방안

구분	예상 장애요인	해결 방안
광학	PDLC 산란이 포화는 줄이지만 영상 구조(Edge/Contrast)도 훼손 가능	Density 보존 지표를 control score에 반영하여 과차광자동 억제. 전면에 시야 안전 하한 MI 강제, VisibilityLoss penalty로 과차광 억제
전력	H-Bridge/LC/PDLC 용량성 부하의 발열·공진·노이즈	계측 기반 보정, 1채널 선검증으로 기본 동작을 이미 확인, 8채널 정격·발열·응답시간은 실측으로 확정
세부 통제	DevKit 핀맵 오류·EN floating·ADC 노이즈·USB 역급전·센서 결측·라이선스	carrier_pinmap 라벨·locking connector, pull-down, RC filter, 역급전 방지, cache·폴백, 사전 라이선스 확인·자체 구현 명시

○ 예상 결과 작품이 활용될 분야 및 방법 제시

- 차량 능동 디밍 글라스 제어 알고리즘·전력부 레퍼런스, 빌트인 캠/ADAS 카메라 전방 광학 상태 보조, 전기차 열관리 보조, 건축·모빌리티 스마트 윈도우 다채널 구동 시스템, 임베디드(영상·센서융합·전력전자·안전제어) 교육 레퍼런스.
- 발전 방향 : 8채널 → 윈도우 분할 매트릭스 확장, 경량 ML 기반 Glare/Thermal 예측, 실차 곡면·면적 스케일업, 내비 경로·도로 기울기 융합.

□ 작품 상세 설명 및 지원 장비 사용 계획

○ 작품 작동 원리 설명

단계	내용
센서·외부 정보 입력	전/후방 카메라가 영상과 AE 메타데이터(노출시간·게인)를 수집하고, VEML7700 4개(전·우·후·좌)가 방향성 조도 벡터를 생성한다. DS18B20은 차량 모형 내부온도를, 날씨 API는 기온·구름량·강수·UV 등 외부 맥락을 공급한다
라즈베리파이 처리	Raspberry Pi 5의 control 서비스가 카메라 ROI별 Saturation Ratio, Mean Brightness, Edge Density를 계산하고, AE 메타데이터와 조도 벡터를 결합해 전면 순간 강광과 360° 광원 방위를 추정한다.
통합 판단	정책 엔진은 차량 상황(주행·정차·차박·주차), 내부온도, 날씨 cache, 방향성 조도, 전면 강광 지표를 하나의 ChannelScore로 통합해 CH0~CH7별 목표 상태(CLEAR/DIM/SCATTER)와 목표 MI(Modulation Index)를 산출한다.
연속 보정	실측 기반 MI-Vrms-투과도 LUT를 inverse로 사용해 목표 MI를 결정하고, MI Servo가 hysteresis·rate limit·low-pass를 적용해 깜빡임을 억제한다. 강한 전면광은 CH0/CH1에 한해 fast-attack으로 즉시 산란시키고, 회복은 제한해 시야 안정성을 유지한다.
실시간 출력	Raspberry Pi가 20Hz JSON Lines로 ESP32-S3 #A/#B에 목표 MI를 전송한다. ESP32는 Fault를 우선 확인한 뒤 16kHz SPWM을 생성하고, Logic Carrier PCB가 CH0~CH7 Power Stage PCB로 신호를 분배한다.
물리 동작	각 Power Stage PCB는 72V DC Link, H-Bridge, LC Filter를 이용해 채널별 0~48Vrms AC 출력을 만들고, 8장 PDLIC의 산란 정도를 독립 제어한다.
상태 피드백	전압·전류·전력부 온도·Fault·채널 MI·카메라 지표가 Raspberry Pi로 되돌아와 StateStore와 CSV 로그에 저장되며, 태블릿 /demo HMI에 3D 차량 상태·ROI overlay·판단 이유·LIVE/REPLAY/MOCK 상태로 표시된다.

-데이터 흐름 : 센서·외부정보 → Raspberry Pi 처리·판단 → ESP32 실시간 구동 → 모형 PDLIC 물리 동작 + 태블릿 상태 표시 구조로 구성한다.

-주요 시연 : ① 360° 손전등(광원 방위 추적) ② 카메라 포화도(산란 미개입↔개입 비교) ③ 뜨거운 여름(날씨·내부온도 기반 열부하 차광) ④ 차박(전 채널 프라이버시)을 동일 제어 엔진으로 실행한다.

○ 작품에 사용될 SW와 HW 설명(라즈베리파이 필수, 지원 장비 사용 방안)

-SW 상세설명(개발 방법 및 활용 기술)

- 라즈베리파이 핵심 처리부 : Raspberry Pi 5(Raspberry Pi OS 64-bit Lite)를 필수 메인 컨트롤러로 사용한다. 영상처리, 센서융합, 날씨 API cache, 제어 정책, UI 백엔드, 상태 저장, CSV 로그를 담당하며 고주파 PWM은 직접 생성하지 않고 ESP32에 위임한다.
- 서비스 구조 : kuglass-control.service와 kuglass-ui.service를 분리한다. UI 장애가 발생해도 제어 루프와 ESP32 명령 송신이 유지되도록 StateStore·CommandQueue·FrameBuffer 구조를 사용한다.
- 영상처리 : Python, OpenCV, libcamera/picamera2를 사용해 전/후방 카메라 프레임과 AE 메타데이터를 동시에 취득한다. ROI별 포화 픽셀 비율, 평균 밝기, Edge Density등을 계산해 전면 순간 강광과 카메라 입력 안정성을 판단한다.
- 센서융합 : VEML7700×4와 TCA9548A I2C 멀티플렉서를 통해 전·우·후·좌 조도를 읽고, 차광 후드 보정값을 적용해 theta_lux와 conf_lux를 산출한다. 전/후방 카메라 highlight cue

와 융합해 360° 광원 방위를 추정한다.

- 외부정보 처리 : 날씨 API의 기온·구름량·강수·UV 정보를 저주기 cache로 관리한다. API 결측 또는 stale 상태에서는 내부온도·조도·카메라 기반 판단으로 폴백한다.
- 정책·제어 알고리즘 : $ChannelScore = W_{mode} \cdot ModeNeed + W_{thermal} \cdot ThermalRisk + W_{dir} \cdot DirectionHit + W_{camera} \cdot FrontInstantGlare + W_{privacy} \cdot PrivacyNeed - W_{visibility} \cdot VisibilityPenalty$ 구조로 채널별 필요 차광도를 계산한다. 전면 CH0/CH1은 카메라·시야 안전 가중치를, CH2~CH7은 방향성 조도·열부하·프라이버시 가중치를 크게 둔다.
- MI 변환·안정화 : 실측 MI-Vrms-투과도 LUT를 구성하고, 목표 투과도에서 MI를 역산한다. hysteresis, rate limit, low-pass, minimum hold time, 전면 fast-attack 예외를 적용해 연속 제어 안정성을 확보한다.
- ESP32 펌웨어 : ESP-IDF 기반으로 USB Serial JSON parser, MCPWM 16kHz SPWM, Channel Manager, ADC task, Fault task, Status TX, Watchdog을 구현한다. 명령 우선순위는 E-Stop > Fault > Manual > Auto로 둔다.
- UI·로그 : Flask + SocketIO + MJPEG 백엔드와 React + TypeScript + Vite + React Three Fiber 기반 /demo 3D HMI를 사용한다. 카메라 Evidence View, ROI overlay, 채널 MI, 판단 이유 문장, 시연 모드 버튼, 수동 제어, 날씨 온도 슬라이더, CSV 로그 및 replay 기능을 제공한다.
- 개발·검증 환경 : macOS에서 VS Code Remote SSH로 Raspberry Pi를 개발하고, VS Code + ESP-IDF로 ESP32 펌웨어를 개발한다. Python venv, requirements.txt, config.yaml, carrier_pinmap.yaml, pytest, mock serial, protocol test vector, HIL 체크리스트를 사용한다.

- HW 상세설명(개발 방법 및 활용 기술)

- 메인·실시간 제어 장치 : Raspberry Pi 5 1대와 ESP32-S3-DevKitC-1 2대를 사용한다. ESP32 #A는 CH0~CH3, #B는 CH4~CH7을 담당해 8채널 실시간 구동 부하를 분산한다.
- 광학 출력부 : 1:10 아이오닉5 기반 모형에 전면 2장, 4도어 4장, 후면 1장, 선루프 1장의 PDLC Smart Film을 부착한다.
- 센서부 : 전방 카메라와 후방 카메라를 장착하고, VEML7700 4개를 전·후·좌·우 방향에 배치한다. 각 조도센서에는 3D 프린팅 차광 후드/폴리메이터를 적용하고, 동일 주소 문제는 TCA9548A I2C 멀티플렉서로 해결한다. 내부온도는 DS18B20으로 측정한다.
- 전력 구동부 : 절연형 DC Adapter와 상용 Boost로 72V DC Link를 생성하고, 채널별 H-Bridge + LC Filter + PDLC 커넥터 구조로 0~48Vrms AC를 출력한다. Carrier-based Simplified Unipolar SPWM의 MI 조절로 중간 산란 상태를 연속 제어한다.
- PCB 구조 : 저전압 Logic Carrier PCB 1장에 ESP32 DevKit socket, CH0~CH7 커넥터, EN_GLOBAL E-Stop, FAULT_N, ADC filter, test point를 배치한다. 고전압 Power Stage PCB는 단일 채널 1종으로 설계하고, 2채널 선검증 후 최종 8장으로 복제한다.
- 센싱·보호 회로 : 채널 전압·전류·전력부 온도 센싱을 적용하고, 과전압·과전류·과열·통신 timeout·ADC 오류·E-Stop 상황에서 ESP32가 Raspberry Pi 판단을 기다리지 않고 로컬 차단한다.
- 전원·안전 구조 : 저전압 로직부와 72V DC Link 고전압부를 물리적으로 분리하고, 메인 퓨즈·채널별 퓨즈·방전저항·HV 표시 LED·E-Stop·절연형 전원·고전압 제어박스 격리를 적용한다.

- 지원 장비 사용 방안

라즈베리파이는 필수 메인 처리 장비로 사용하고, 태블릿은 /demo HMI 표시 및 시연 모드 입력 장치로 사용한다. 고휘도 LED/손전등은 역광·측면광·후방광·상부 일사 시연용 입력으로 사용

한다.

이외 PCB 제작비, 필라멘트, 안전장비, 전자부품, 시연 환경 구성 등에 대한 지원을 받을 수 있다.

○ 기술 공부 내용 및 적용 방안

기술 공부 내용	적용 방안
단상 H-Bridge SPWM·MI 제어	PDLC 용량성 부하 대응 LC 설계, MI제어 기반 PDLC 0~48Vrms 연속 제어
PDLC 전기-광학 특성	MI-Vrms-투과도 LUT 작성, 비선형 보정
OpenCV ROI·포화·Edge + libcamera AE 메타데이터	전면 순간 강광 추정, fast-attack·시야 안전 하한
I2C 멀티플렉싱·조도 벡터 추정	VEML7700 4방향 360° 광원 방위 추정
FreeRTOS/ESP-IDF 실시간·Fault 처리	10ms 로컬 차단, 채널 격리, 안전 상태 머신

□ 개발 일정

[illegible]

□ 팀 구성 및 역량

No	구분	성명	팀 내 담당 업무	업무 관련 역량(개발 언어, 프로젝트 경험 등)
1	팀장	최무영	- 프로젝트 일정·예산·문서·발표 총괄 - Raspberry Pi 기반 영상 분석 및 센서융합 SW 제작 - Flask/SocketIO 백엔드 및 React 기반 태블릿 UI 구현 - 카메라 영상 KPI(포화 감소율·Edge 보존율) 측정	- 개발 언어 및 기술스택 -Python, JavaScript(React), C - 프로젝트 경험 - Raspberry Pi. OpenCV 기반 영상처리 경험 - 교내 정보운영팀 학생개발팀 프론트엔드 근무, 교내 웹,앱 서비스 개발 경험 - LG aimers 8기 LLM 양자화 프로젝트 - LangGraph 기반 제조기업 업무 자동화 외주 프로젝트 경험
2	팀원	전수찬	1:10 아이오닉5 3D 프린팅 모형 설계·조립 - 전력부 제어박스 외함 설계·제작 - 웹 UI용 3D 디지털 트윈 GLB 모델 제작 - 시연 영상 기획 및 제작	- 개발 언어 및 기술스택 - Python, Fusion 360, Blender - 프로젝트 경험 - 3D 모델링을 통한 시각화 자료 제작 및 시연 환경 구축 경험. - 3D 프린터를 활용한 시제품 제작 및 조립 경험 - 전자부품 배선, 납땜, 하네스 제작 경험
3	팀원	김덕렬	- ESP32-S3 2대 기반 실시간 제어 펌웨어 구현 및 각종 테스트 진행. 16kHz SPWM 생성 및 MI 기반 출력 전압 제어 - Raspberry Pi-ESP32 통신 및 Fault injection 검증	- 개발 언어 및 기술스택 - C/C++기반 제어 알고리즘 구현 능력 - 프로젝트 경험 - ESP-IDF 기반 펌웨어 개발 경험 - MCPWM/PWM 기반 모터·전력 제어 경험 - PSIM, PSpice 등을 이용하여 전력전자 시스템 검증 경험.
4	팀원	이승민	- PDLC 전력 구동 회로, Logic Carrier PCB 설계 및 제작 - 전원·안전 회로 설계 및 제작 - PCB 초기 구동 검증 및 전기·열 계측 - 전기적 KPI(출력 파형·발열·Fault 응답 시간) 검증	- 개발 언어 및 기술스택 - Python, LTspice, PSIM - 프로젝트 경험 - LTspice, PSIM 등을 활용하여 전력전자 회로 시뮬레이션 및 설계 최적화 수행 경험 - Python을 활용해 데이터 분석 및 전기적 KPI 도출 프로젝트 수행 경험 - 전력전자회로를 직접 설계하고 출력 결과를 바탕으로 회로 성능 분석 및 개선 경험 - KiCad 및 Altium을 활용하여 회로도 작성 및 PCB 레이아웃 설계 경험